

CSCE 671 HUMAN-COMPUTER INTERACTION · MODULE 01 · LECTURE NOTES

Why Interfaces Fail

The gap between the system's model and the user's.

MODULE OUTCOMES

- Explain the gulfs of execution and evaluation.
- Explain why designers are blind to their own designs.
- Explain affordances and signifiers and distinguish them.
- Explain why 'user error' is usually a design finding.
- Diagnose a failure mechanically rather than descriptively.

1 Two gulfs

THE GULF OF EXECUTION IS NOT KNOWING WHAT TO DO; THE GULF OF EVALUATION IS NOT KNOWING WHAT HAPPENED

The gulf of execution: the person has a goal and cannot work out which available action achieves it — which is a problem of what the interface *shows is possible* (§3's signifiers).

The gulf of evaluation: the person acted and cannot tell what the system actually did — which is a problem of feedback, and **is the one more often neglected**, because feedback feels like a detail while the controls feel like the design.

And they fail differently, observably. An execution gap produces hesitation, hovering, and searching; an evaluation gap produces repeated actions, undo attempts, and eventually a settled wrong belief about how the system works — which then causes further errors downstream.

So the diagnosis matters practically: adding a label fixes an execution gap and does nothing at all for an evaluation gap — and **confusing the two is why interface fixes frequently fail to help**, which is a result you can observe in your own redesign in Project 2.

Mechanism	What the user experiences	Where it is treated
Missing or misleading signifier	Does not know the action is possible, or believes a different one is.	Module 01 §3
Absent or ambiguous feedback	Cannot tell what happened after acting.	Module 01 §1
Perceptual failure	Did not see it at all — contrast, position, size, or grouping.	Module 02

Mechanism	What the user experiences	Where it is treated
Attention failure	Saw it and did not process it — which is a different and more common failure.	Module 03 §1
Memory demand	Had to hold something in mind across steps and did not.	Module 03 §2
Mental model mismatch	Expected the system to work in some other way entirely.	Module 04

This table is the diagnostic vocabulary of the course, and **Project 1** requires every observed failure to be attributed to one of these mechanisms rather than described as 'confusing'. The reason is practical: 'confusing' admits no fix, and each of these six mechanisms has a known family of fixes — so the attribution is what converts an observation into an action.

2 Designer blindness

YOU CANNOT UN-KNOW HOW YOUR OWN INTERFACE WORKS, WHICH MAKES YOUR JUDGEMENT OF IT NEARLY WORTHLESS

You know the conceptual model, the vocabulary, the state machine, the intended order of operations, and which of the three similar-looking buttons does the thing — and none of that is available to the user, who has only the pixels.

So the interface is legible to you by construction — you built the legibility in by knowing it — **and you cannot simulate not knowing, which is a cognitive limit rather than a failure of imagination or empathy** and is why trying harder does not help.

Which means your opinion about your own interface's usability is nearly worthless, and the opinions of the colleagues who built it with you are no better — they share the model, which is exactly the thing that disqualifies them.

And the only remedy is observation (Module 07) — watching five people struggle with it is worth more than any amount of reasoning about it, and this is the single most important claim in the course. Everything in Modules 02 through 05 helps you predict and diagnose; only Module 07 tells you what actually happens.

- **Design arguments cannot be settled by argument — two designers disagreeing about what is obvious are both unreliable witnesses, and the resolution is to watch somebody** rather than to escalate. This is liberating as well as humbling: it ends unresolvable meetings.
- **Internal demos tell you nothing about usability. Everybody in the room knows the model, so the demo tests the implementation and not the design —** which is a useful thing to do and is a different thing.
- **And a user who 'does not read' is not the finding. Nobody reads —** it is a stable, repeatedly measured property of how people use software (Module 03 §1) — **and a design that relies on reading has made an assumption it cannot support.**

- So **'the user should have...' is never a conclusion, because the user is not available to be changed** and the design is. This is the same move as CSCE 701 Module 11's compliance budget: the person is a fixed constraint and the system is the variable.
- **Which makes observation a method rather than a courtesy. 'The user should have read it' is the sentence that ends an investigation prematurely — and the measured non-reading rate is the reason it is not a defence,** in design review or anywhere else.

3 Affordances and signifiers

AN AFFORDANCE IS WHAT AN OBJECT PERMITS; A SIGNIFIER IS WHAT TELLS YOU SO

A door affords pushing whether or not anything indicates it — the affordance is a relation between the object's properties and the person's capabilities, and it exists whether or not it is perceived.

A flat plate is a signifier: it communicates that pushing is the thing to do — and a handle fitted to a push door is a signifier for the wrong affordance, which is precisely why those doors fail and why they are the canonical example.

So most interface failures are missing or misleading signifiers rather than missing affordances — the action was entirely possible and nothing indicated it, or something indicated something else.

Which is why the distinction is worth keeping straight: 'add an affordance' is usually a confused request (the affordance is already there), and 'add a signifier' is an actionable one — and the sloppy usage of 'affordance' to mean both has made a good deal of design discussion less precise than it could be.

- **Hidden gestures and keyboard shortcuts** — the affordance exists and nothing indicates it, **so only people who were told will ever find it**, which makes the feature effectively private.
- **Elements that look clickable and are not, and elements that are clickable and do not look it — both of which cost trust in every other element** on the screen, because the user can no longer rely on appearance.
- **Disabled controls with no explanation**, which **signify that something is possible while refusing it, with no indication of what would make it possible** — a combined execution and evaluation gap, and one of the most frustrating patterns there is.
- **And inconsistent signifiers within a single system — the same visual treatment meaning different things in different places, which destroys the learning the user has already done.**
- **Which is why consistency is worth more than local optimality** (Module 05 §2): a slightly worse treatment used everywhere beats a better one used inconsistently. **Inconsistency is expensive because it invalidates learning — a user who learned your convention and then found an exception now trusts none of it**, and has to check everything.

4 User error

“USER ERROR” NAMES A DESIGN FINDING YOU HAVE DECIDED NOT TO INVESTIGATE

People make mistakes at a predictable and well-measured rate (Module 03 §3), and a design whose correct operation depends on their not doing so has made an assumption that does not hold — which is a defect in the design rather than in the person.

So the question is never 'why did they do that' but 'why did the system permit it to matter' — which is exactly CSCE 701 Module 10 §4's blameless post-mortem framing, arriving in this course first chronologically and with the same justification: the first question produces a defensive answer and the second produces a list of fixes.

And the fixes are the same shape as there: make the error impossible, make it recoverable, or make it visible — in that order of preference, because each requires less of the user than the last.

Which means confirmation dialogues are the weakest available fix — they are read once and then dismissed reflexively, so they stop functioning after the second time — and undo is the strongest, because it requires nothing of the user at the moment of the error, which is precisely the moment they have no attention to spare.

- **Modules 02 to 05 are what people are actually like**, and what follows for design — perception, attention, memory, error, mental models, and the interaction principles.
- **Modules 06 to 09 are method** — prototyping at the right fidelity, usability testing, controlled experiments with people, and qualitative observation.
- **Modules 10 and 11 are specific domains** — interfaces for programmers (which is the interface design most of this program's graduates will actually do), and the input and output modalities.
- **And Modules 12 and 13 are consequences and honest claims** — deliberately manipulative design, and what a usability study establishes.
- **With Module 07 being the one to internalise**, because everything else in the course is argument and that is evidence. If you take one thing from this course, take the habit of watching five people — it is cheap, it is uncomfortable, and it is the only reliable source of information available in this subject.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Norman — The Design of Everyday Things, chapters 1 and 2	§1, §3, and §4 — the gulfs, the affordance/signifier distinction, and the human-error argument, from the source.
Nielsen — the usability heuristics, and the articles on scanning (free)	§2's non-reading finding and §3's signifier problems, with the studies behind them.
Norman — Affordances and Design (free)	§3's distinction, clarified by its author — written specifically because the term had been misused.

Resource	What it covers for this module
Woods & Cook — on human error in complex systems	§4's argument from safety engineering — the intellectual source of the blameless framing. Library copy.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Find a door you have pushed when you should have pulled and identify the misleading signifier.
- 2 Take one interface failure you have experienced and classify it as an execution or an evaluation gap.
- 3 Classify ten failures against Part 1's six mechanisms.
- 4 Explain a feature of your own software to someone unfamiliar and note every piece of context you had to supply.
- 5 Watch one person use something you built, without helping.
- 6 Count the things they did not read.
- 7 Find a hidden affordance in software you use daily.
- 8 Find an inconsistent signifier within one application.
- 9 Find a confirmation dialogue you dismiss reflexively and design the undo that would replace it.
- 10 Rewrite one 'user error' report as a design finding.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Distinguish the two gulfs and say how each fails observably.
- 2 Why does the diagnosis determine whether a fix helps?
- 3 Name the six failure mechanisms.
- 4 Why is your judgement of your own interface unreliable?
- 5 Why are internal demos uninformative about usability?
- 6 Why is 'the user did not read' not a finding?
- 7 Distinguish an affordance from a signifier, and say which is usually missing.
- 8 Why is inconsistency expensive?
- 9 Reframe 'user error', and give the three fixes in order.
- 10 Why is a confirmation dialogue weak and undo strong?